Datasets and Feature Extraction

Companion to *"Round Trip Time: A Benign Signal or an Indirect Window into Datacenter Workloads?"*

This document accompanies the paper *"Round Trip Time: A Benign Signal or an Indirect Window into Datacenter Workloads?"* It contains all RTT measurement datasets and the feature-extraction code used to turn raw RTT probe traces into the per-window feature CSVs that drive every downstream analysis. The contents are self-contained: a fresh user can rebuild every CSV in `features_allpaths/` from the raw JSON in `raw_rtt_phase*/` using the two scripts shipped here (`extract_features_v3_fixed.py` and `extract_all.py`).

This document does **not** describe model training or evaluation — it covers only the data and the feature pipeline.

Contents

1 Folder Layout **2**

2 Path Symbols and Naming **2**

3 Workloads (15 classes) **3**

4 Per-Phase Datasets **4**

4.1 Phase 1 — `raw_rtt_phase1/` . . . 4

4.2 Phase 2 — `raw_rtt_phase2/` . . . 4

4.3 Phase 3 — `raw_rtt_phase3/` . . . 5

4.4 Phase 4 — `raw_rtt_phase4/` . . . 5

4.5 Phase 5 — `raw_rtt_phase5/` . . . 5

4.6 Phase 6 — `raw_rtt_phase6/` . . . 6

5 Raw RTT JSON Format **6**

6 Feature CSV Format (`features_allpaths/`) **7**

6.1 Metadata Columns . . . 7

6.2 Approximate Row Counts . . . 8

7 Feature Extraction Pipeline **8**

7.1 Pipeline (per trace file) . . . 8

7.2 Path / Run Table 9
7.3 Rebuilding the CSVs 9

8 Feature Catalogue (104 features) 9
8.1 Amplitude Structure and Variability of Congestion (20) 10
8.2 Temporal Burst and Event Patterns (35) 11
8.3 Multi-Scale Variability Across Time (23) 12
8.4 Periodicity and Frequency Structure (19) 13
8.5 Long-Range Dependence and Persistence (2) 14
8.6 Temporal Complexity in Low-Intensity Signals (5) 14
8.7 Total 14

1 Folder Layout

```
datasets/
|-- README.md, README.tex                    <- this document
|-- extract_features_v3_fixed.py             <- feature definitions
   (104 features)
|-- extract_all.py                           <- driver: walks every
   phase/path
|
|-- raw_rtt_phase1/                          <- per-run RTT JSON
|-- raw_rtt_phase2/
|-- raw_rtt_phase3/
|-- raw_rtt_phase4/
|-- raw_rtt_phase5/
|-- raw_rtt_phase6/
|
\-- features_allpaths/                       <- per-window feature
   CSVs
```

- **`raw_rtt_phaseN/`** — one JSON file per run. This is the format the feature extractor reads.
- **`features_allpaths/`** — output of running `extract_all.py`. One CSV per *path symbol* (see §2), one row per 60 s window.

2 Path Symbols and Naming

Throughout the project, every measurement run is tagged with a short path symbol. The convention is:

- Symbols starting with **A** are *victim* paths. They are reserved for evaluation only and are never used to fit any preprocessing or training stage.
- Symbols starting with **B / C / E′ / F / P5p** are *profile* paths used as training data.

Symbol	Phase	Role	Description
A	1	victim	Phase-1 victim path (primary cross-path test set)
B	1	profile	Phase-1 profile path
A2	2	victim	Phase-2 victim path (re-collected, clean as of 2026-04-10)
B2	2	profile	Phase-2 profile path; iperf workloads route through this path (fixes earlier STP re-route)
C	3	profile	Path-C profile (path-diversity training data)
E′ (Ep)	3	profile	Path-E′ profile
F	3	profile	Path-F profile
P5p	5	profile	Phase-5 profile path. Phase-5 victim (P5v) was dropped due to measurement issues and is not used.
A4	4	victim	Mixed-workload pairs (victim path only)
A6v1	6	victim	Parameter variation, *lighter* parameters (≈2–4× smaller than baseline)
A6v2	6	victim	Parameter variation, *comparable / heavier* parameters than baseline

The path symbol shows up in three places: (1) the phase folder it lives in (`raw_rtt_phaseN/`), (2) the filename pattern (see per-phase tables below), and (3) the `path` column inside every feature CSV.

3 Workloads (15 classes)

Each phase exercises the same 15-workload catalogue, organised in three families.

#	Workload	Family	Generator
1	`allreduce`	distributed	PyTorch Gloo collective
2	`pipeline`	distributed	PyTorch Gloo collective
3	`shuffle`	distributed	PyTorch Gloo collective
4	`scatter_gather`	distributed	PyTorch Gloo collective
5	`federated`	distributed	PyTorch Gloo collective
6	`checkpoint`	client-server	TCP
7	`preprocess`	client-server	TCP
8	`db_queries`	client-server	TCP
9	`inference`	client-server	TCP
10	`log_agg`	client-server	TCP
11	`microservice`	client-server	TCP
12	`dfs_reads`	client-server	TCP
13	`bulk`	bandwidth	iperf3
14	`storage_rep`	bandwidth	iperf3
15	`migration`	bandwidth	iperf3

Five of these — `db_queries`, `inference`, `log_agg`, `microservice`, `shuffle` — produce RTT perturbations close to measurement noise. They are sometimes merged into a single *light_traffic*

class for an alternative 11-class evaluation, but the raw datasets always use the full 15-class label set.

4 Per-Phase Datasets

4.1 Phase 1 — `raw_rtt_phase1/`

Property	Value
Paths	A (victim), B (profile)
Workloads	15
Runs / workload	20
Trace duration	300 s
Total traces	600 (15 × 20 × 2)
Status	Clean — primary dataset

Filename patterns:

```
victim_<workload>_run<N>_rtt.json      <- path A (victim)
profile_<workload>_run<N>_rtt.json     <- path B (profile)
```

where `N` ∈ 1..20.

4.2 Phase 2 — `raw_rtt_phase2/`

Property	Value
Paths	A2 (victim), B2 (profile)
Workloads	15
Runs / workload	5
Trace duration	600 s
Status	Clean (re-collected 2026-04-10)

Filename patterns:

```
victim_<workload>_run<N>_p2_rtt.json
profile_<workload>_run<N>_p2_rtt.json
```

where `N` ∈ 1..5.

Note: Phase 2 was re-collected because the original Phase-2 iperf workloads (`bulk`, `storage_rep`, `migration`) had near-zero signal due to an STP re-route that bypassed the link-rate-limited spine. In the clean recollection, iperf workloads route through a separate profile path. Only the clean recollection is included in this folder.

4.3 Phase 3 — `raw_rtt_phase3/`

Property	Value
Paths	C, E′ (Ep), F — all profile-side
Workloads	15
Runs / workload	5 each path
Trace duration	300 s
Status	Clean — adds path-diversity to the training pool

Filename patterns:

```
pathC_<workload>_run<N>_rtt.json
pathEp_<workload>_run<N>_rtt.json
pathF_<workload>_run<N>_rtt.json
```

where N ∈ 1..5.

4.4 Phase 4 — `raw_rtt_phase4/`

Property	Value
Paths	A4 (victim, mixed-workload pairs)
Pairs	8
Runs / pair	5
Total traces	40
Status	Test-only (used for OOD analysis)

Each Phase-4 run runs a **primary** workload and an **interferer** workload simultaneously on the victim path. The filename encodes both:

Filename pattern:

```
victim_<primary>_vs_<interferer>_run<N>_p4_rtt.json
```

where N ∈ 1..5.

The 8 pairs combine 7 primary workloads with two interferers (`bulk` and `checkpoint`). Phase 4 is included for completeness; it is intended only as a held-out, mixed-workload stress test.

4.5 Phase 5 — `raw_rtt_phase5/`

Property	Value
Paths	P5p (profile only)
Workloads	15
Runs / workload	5
Trace duration	600 s
Status	Profile side included in training pool. Phase-5 victim (P5v) dropped due to measurement issues.

Filename pattern:

```
profile_<workload>_run<N>_p5_rtt.json
```

where N ∈ 1..5. Although a P5 victim was originally collected, it is not used in any analysis and is not included in this folder.

4.6 Phase 6 — `raw_rtt_phase6/`

Property	Value
Paths	A6v1, A6v2 (victim only)
Workloads	15
Variations	2 (v1 = lighter, v2 = heavier)
Runs / (workload × variation)	3
Trace duration	600 s
Status	Test-only (parameter-generalisation)

Filename pattern:

```
victim_<workload>_v1_run<N>_p6_rtt.json    <- lighter parameters
victim_<workload>_v2_run<N>_p6_rtt.json    <- heavier parameters
```

where N ∈ 1..3.

Phase 6 stresses workloads **outside** the parameter ranges seen during training:

- `v1`: each workload's intensity-controlling parameters (e.g. number of streams, message size, request rate) are scaled down by ∼2–4×.
- `v2`: each workload's parameters are at least as heavy as the training baseline; some are made noticeably heavier.

The `preprocess` workload was re-collected on 2026-04-12 after a probe packet-loss issue was fixed; the fixed traces are the ones included in `raw_rtt_phase6/`.

5 Raw RTT JSON Format

Every file in `raw_rtt_phase*/` is a JSON list of probe records, in send order:

```
[
  {"seq": 0, "send_ts": 1774027367.0588, "rtt_ms": 167.444},
  {"seq": 1, "send_ts": 1774027367.2263, "rtt_ms": 166.781},
  {"seq": 2, "send_ts": 1774027367.3930, "rtt_ms": 166.766},
  ...
]
```

Field	Type	Meaning
`seq`	int	Monotonic probe sequence number assigned by the prober
`send_ts`	float	Send timestamp, Unix epoch seconds (sub-millisecond resolution)
`rtt_ms`	float	Measured round-trip time in milliseconds

Probing parameters (identical across phases):

- UDP echo on port 8888.
- Synchronous, RTT-bounded — the next probe is sent only after the previous reply arrives, so the effective rate is ∼10 Hz on a 100 ms-RTT path.
- One probe stream per run; the prober runs end-to-end across the entire trace duration.

Special values:

- `rtt_ms = -1` is a **probe-timeout sentinel**, meaning no reply was received within the prober's timeout. This appears mainly in Phase 6 V1 (`scatter_gather` most often). The feature extractor maps these to `NaN` and filters them out before windowing.

6 Feature CSV Format (`features_allpaths/`)

Running `extract_all.py` produces one CSV per path symbol:

```
features_allpaths/<PATH>_features_fixed.csv
```

Each row corresponds to a **single 60 s window** of a single run. Every CSV has **109 columns**: 104 features followed by 5 metadata columns.

6.1 Metadata Columns

These five columns are always the **last** columns in the CSV:

Column	Meaning
`label`	Workload name (one of the 15 strings above)
`run`	Run index (1-based, as it appears in the filename)
`window_start`	Start time of the window in seconds, relative to the start of that run (0, 10, 20, . . .)
`path`	Path symbol (e.g. `A`, `B`, `A6v2`)
`baseline_rtt`	Median RTT (in ms) of the *entire run*. Used to normalise the window before feature extraction.

(`label`, `run`, `path`) uniquely identifies a run; (`label`, `run`, `path`, `window_start`) uniquely identifies a row.

A single companion file is also provided:

- `A4int_features_fixed.csv` — same Phase-4 mixed-workload data as `A4`, but each window is labelled with the *interferer* workload instead of the primary.

6.2 Approximate Row Counts

Path	Runs / wl	Trace s	Windows / run	Approx rows
A	20	300	25	7 500
B	20	300	25	7 500
A2	5	600	55	4 125
B2	5	600	55	4 125
C	5	300	25	1 875
Ep	5	300	25	1 875
F	5	300	25	1 875
P5p	5	600	55	4 125
A6v1	3	600	55	2 475
A6v2	3	600	55	2 475

(Windows / run = (duration − 60)/10 + 1. Actual counts are slightly lower because windows with fewer than 20 valid samples are dropped.)

7 Feature Extraction Pipeline

The extraction code is a **two-file** pipeline:

File	Role
`extract_features_v3_fixed.py`	Defines all 104 features. Pure function: `extract_window_features(normed_rtt, sample_rate) -> dict`.
`extract_all.py`	Driver. Iterates every path × workload × run, slides the window, calls the extractor, writes the CSV.

All thresholds inside the extractor (run-length, spike/elevation) are **percentile-based** on the window's own distribution. There are no hardcoded absolute RTT thresholds, which keeps the feature distribution stable across paths with different baseline noise floors.

7.1 Pipeline (per trace file)

The driver `extract_all.py` does the following for every JSON file:

```
1. Load the JSON  -> arrays (ts, rtt) of equal length
2. Normalise:
       baseline = median(rtt[rtt > 0])              # baseline_rtt
          column
       normed   = (rtt - baseline) / baseline       # NaN where rtt
          <= 0
3. Drop NaNs (probe-timeout sentinel rtt_ms = -1 maps to NaN)
4. Estimate sample_rate ~ 1 / median(diff(ts))      # ~10 Hz
5. Slide a 60s window with 10s stride over the run:
       for each window: if at least 20 valid samples, call
                        extract_window_features(window, sample_rate)
6. Append the 5 metadata fields (label, run, window_start, path,
   baseline_rtt)
7. Stack rows, write features_allpaths/<PATH>_features_fixed.csv
```

Constants:

Constant	Value	Where
`WINDOW_SEC`	60 s	window length
`STRIDE_SEC`	10 s	window stride
Min valid samples	20	windows below this are skipped

7.2 Path / Run Table

`extract_all.py` hardcodes the (path → folder, filename pattern, run-range) map:

Path	Folder	Filename template	Runs
A	`raw_rtt_phase1`	`victim_{wl}_run{run}_rtt.json`	1–20
B	`raw_rtt_phase1`	`profile_{wl}_run{run}_rtt.json`	1–20
A2	`raw_rtt_phase2`	`victim_{wl}_run{run}_p2_rtt.json`	1–5
B2	`raw_rtt_phase2`	`profile_{wl}_run{run}_p2_rtt.json`	1–5
C	`raw_rtt_phase3`	`pathC_{wl}_run{run}_rtt.json`	1–5
Ep	`raw_rtt_phase3`	`pathEp_{wl}_run{run}_rtt.json`	1–5
F	`raw_rtt_phase3`	`pathF_{wl}_run{run}_rtt.json`	1–5
P5p	`raw_rtt_phase5`	`profile_{wl}_run{run}_p5_rtt.json`	1–5
A6v1	`raw_rtt_phase6`	`victim_{wl}_v1_run{run}_p6_rtt.json`	1–3
A6v2	`raw_rtt_phase6`	`victim_{wl}_v2_run{run}_p6_rtt.json`	1–3

Phase 4 is extracted separately by a dedicated script (`extract_phase4.py` in the project root), because Phase 4 uses a `<primary>_vs_<interferer>` filename schema rather than the single-workload schema above.

7.3 Rebuilding the CSVs

From the project root (where `extract_features_v3_fixed.py` and `extract_all.py` also live as the canonical copies), run:

```
# rebuild every CSV
python3 extract_all.py

# rebuild a subset
python3 extract_all.py A A2 A6v2
```

Output goes to `features_allpaths/`. A log of per-workload window counts and dropped-NaN counts is written to `extract_all_log.txt`.

8 Feature Catalogue (104 features)

All features are computed on the **normalised** signal

$$x = \frac{\text{rtt} - \text{baseline}}{\text{baseline}}$$

after dropping NaNs. Below, `valid` denotes the within-window non-NaN samples, n their count, std_val = std(`valid`) and `sample_rate` is estimated per run from inter-probe intervals ($\sim$10 Hz).

The 104 features are grouped into the six categories used in the paper. Counts in the headings sum to 104.

8.1 Amplitude Structure and Variability of Congestion (20)

Captures the magnitude, spread and tail behaviour of the congestion-induced RTT deviation within the window.

Basic shape (7)

Feature	Definition
`cv`	std(`valid`)/(mean(|`valid`|) + 10^{-10}) — coefficient of variation
`iqr`	75th − 25th percentile
`skew`	Sample skewness (`scipy.stats.skew`)
`kurtosis`	Excess kurtosis (`scipy.stats.kurtosis`)
`range`	max − min (`np.ptp`)
`p95_p50_ratio`	95th − 50th percentile (despite the name, this is a difference)
`p99_p50_ratio`	99th − 50th percentile

Spike / elevation (3)

spike_thresh = percentile(`valid`, 90); elevated = `valid` > spike_thresh.

Feature	Definition
`elevated_frac`	mean(elevated)
`mean_spike_amp`	Mean amplitude of elevated samples (0 if none)
`max_spike_amp`	max(`valid`)

Histogram bins on z-scored signal (5)

z = `valid`/std_val (or zero if std_val ≈ 0). For each bin [lo, hi) the feature is the fraction of samples that fall into it.

Feature	Bin
`bin_near_zero`	$[-0.5, 0.5)$
`bin_moderate_pos`	$[0.5, 2.0)$
`bin_moderate_neg`	$[-2.0, -0.5)$
`bin_large_pos`	$[2.0, 5.0)$
`bin_extreme_pos`	$[5.0, \infty)$

Energy quarters (4)

Sum-of-squares within each consecutive quarter of the window, normalised by total window energy. e_1, e_2 are the energies of the first and second halves.

Feature	Definition
`energy_q1`	$\sum \texttt{valid}[0 : n/4]^2 \,/\, (e_1 + e_2 + 10^{-15})$
`energy_q2`	$\sum \texttt{valid}[n/4 : n/2]^2 \,/\, (e_1 + e_2 + 10^{-15})$
`energy_q3`	$\sum \texttt{valid}[n/2 : 3n/4]^2 \,/\, (e_1 + e_2 + 10^{-15})$
`energy_q4`	$\sum \texttt{valid}[3n/4 : n]^2 \,/\, (e_1 + e_2 + 10^{-15})$

Energy asymmetry (1)

Feature	Definition
`energy_asymmetry`	$(e_2 - e_1)/(e_1 + e_2 + 10^{-15})$, where e_1, e_2 are sum-of-squares of the first and second halves

8.2 Temporal Burst and Event Patterns (35)

Captures discrete congestion events: how often the signal sits above noise, how long elevated runs last, how often probes time out, and how rapidly the signal changes between consecutive samples.

Probe timeout (1)

Feature	Definition
`timeout_frac`	Fraction of probe slots in the window that were dropped ($1-n/n_{\text{total}}$). Captures probe-timeout pressure under heavy congestion.

Burst characterisation (7)

A *burst* is a contiguous run of elevated samples (using the same `elevated` mask as in the amplitude category). WINDOW_SEC = 60.

Feature	Definition
`burst_count`	Number of bursts
`mean_burst_len`	Mean burst length, in seconds (samples / `sample_rate`)
`std_burst_len`	Std of burst length (s)
`cv_burst_len`	std/mean of burst length
`mean_ibi`	Mean inter-burst interval (s); `WINDOW_SEC` if fewer than 2 bursts
`cv_ibi`	std/mean of inter-burst intervals
`burst_rate`	burst_count/WINDOW_SEC

Run-length statistics (24)

Four **percentile-based** thresholds on |`valid`|:

Threshold	Percentile	Rough sigma equivalent
`tiny`	60	$\sim 0.5\,\sigma$
`small`	75	$\sim 1.0\,\sigma$
`med`	90	$\sim 2.0\,\sigma$
`large`	97	$\sim 3.0\,\sigma$

For each threshold t, six features are emitted (24 total):

Feature	Definition
`rl_<t>_max`	Longest run of `valid` $>$ thresh
`rl_<t>_mean`	Mean run length
`rl_<t>_count`	Number of runs
`rl_<t>_cv`	std/mean of run lengths (0 if ≤ 1 run)
`rl_<t>_quiet_max`	Longest run of `valid` $\leq$ thresh
`rl_<t>_quiet_mean`	Mean quiet-run length

First-order temporal dynamics (3)

Computed on diffs = diff(`valid`).

Feature	Definition
`mean_abs_diff`	mean(|diffs|)
`diff_skew`	Skewness of `diffs`
`diff_kurtosis`	Kurtosis of `diffs`

8.3 Multi-Scale Variability Across Time (23)

Captures how the variance of the signal changes when it is observed at different temporal resolutions, from sub-second chunks up to halves of the window.

Multi-scale chunk variability (6)

The window is split into chunks of three sizes; per-chunk means and stds are aggregated. For each scale $s \in$ {fine: 20 chunks, mid: 6, coarse: 2}:

Feature	Definition
`inter_chunk_cv_<s>`	std(chunk_means)/(mean(|chunk_means|) + 10^{-10})
`mean_intra_chunk_cv_<s>`	mean(chunk_stds)/(std(`valid`) + 10^{-10})

$\rightarrow 2 \times 3 =$ **6** features (`fine`, `mid`, `coarse`).

Variance-ratio scale features (3)

Variances of fine chunks ($n//20$) and coarse chunks ($n//4$):

Feature	Definition
`var_ratio_fine_coarse`	mean(fine_vars)/(mean(coarse_vars) + 10^{-15})
`var_cv_fine`	std/mean of fine-chunk variances
`var_cv_coarse`	std/mean of coarse-chunk variances

Wavelet (14)

A 4-level Haar-style lifting decomposition is applied directly in NumPy (no `pywt` dependency). Total energy of the input is used to normalise the energy ratios. For each level $L \in \{1, 2, 3, 4\}$:

Feature	Definition
`wavelet_energy_d<L>`	Detail-coefficient energy at level L, normalised by total energy
`wavelet_std_d<L>`	Std of detail coefficients at level L
`wavelet_max_d<L>`	Max absolute detail coefficient at level L

Plus the residual approximation:

Feature	Definition
`wavelet_energy_approx`	Energy of the level-4 approximation, normalised
`wavelet_std_approx`	Std of the level-4 approximation

$\rightarrow 4 \times 3 + 2 = \mathbf{14}$ features.

8.4 Periodicity and Frequency Structure (19)

Captures repeating patterns and frequency content of the RTT signal — both via direct autocorrelation at known lags and via the magnitude spectrum.

Frequency-domain / FFT (10)

Computed only when $n \geq 32$. Workflow: detrend $\rightarrow$ Hann window $\rightarrow$ real FFT $\rightarrow$ exclude DC bin $\rightarrow$ normalise.

Feature	Definition
`spectral_entropy`	Shannon entropy of normalised magnitude spectrum, divided by $\log_2(\text{len})$ (so $\in [0, 1]$)
`dominant_freq`	Frequency of the largest bin (Hz)
`dominant_freq_power`	Normalised magnitude of the largest bin
`top3_freq_power`	Sum of normalised magnitudes of the three largest bins
`band_power_very_low`	$\sum$ normalised power in 0.001–0.05 Hz
`band_power_low`	$\sum$ normalised power in 0.05–0.20 Hz
`band_power_mid`	$\sum$ normalised power in 0.20–1.0 Hz
`band_power_high`	$\sum$ normalised power in 1.0–3.0 Hz
`spectral_centroid`	$\sum f \cdot p / \sum p$
`spectral_rolloff`	Lowest frequency below which 85 % of cumulative power lies

Autocorrelation at fixed lags (7)

For each lag_sec $\in \{0.5, 1, 2, 5, 10, 20, 30\}$ (in seconds), with lag = round(lag_sec×sample_rate):

Feature	Definition
`ac_lag_<lag_sec>s`	corrcoef(`valid`$[: -\text{lag}]$, `valid`$[\text{lag} :]$) (0 if non-finite or lag $\geq n - 1$)

$\rightarrow$ **7** features.

Autocorrelation decay (2)

The autocorrelation is computed at lags $1..\min(n/2, 50)$.

Feature	Definition
`ac_decay_rate`	ac[0] − ac[4] (drop in correlation over 5 lags)
`ac_half_life`	First lag (in seconds) where ac $< 0.5 \cdot$ ac[0]; if never, the maximum lag computed (in seconds)

8.5 Long-Range Dependence and Persistence (2)

Captures whether the signal has memory across long time scales — i.e. whether deviations decay quickly or persist throughout the window.

Feature	Definition
`hurst`	Hurst exponent via R/S analysis over up to 7 dyadic scales (returns 0.5 on too-short windows)
`trend_slope`	Slope of a linear fit `valid` $\sim t/n$, divided by std_val$+10^{-15}$

8.6 Temporal Complexity in Low-Intensity Signals (5)

Captures the *unpredictability* of the signal. These features remain informative when the workload produces only small RTT deviations: low-amplitude signals can still differ in how regular or random their fluctuations are.

Feature	Definition
`approx_entropy`	Approximate entropy ($m = 2$, $r = 0.2\times$std); window capped at 300 samples for cost
`sample_entropy`	Sample entropy ($m = 2$, $r = 0.2 \times$ std); same 300-sample cap
`perm_entropy_d1`	Permutation entropy, order = 3, delay = 1, normalised by $\log_2(3!)$
`perm_entropy_d2`	. . . delay = 2
`perm_entropy_d5`	. . . delay = 5

8.7 Total

Category	Sub-group	Count
Amplitude structure and variability of congestion	basic shape (7) + spike/elevation (3) + z-scored hist bins (5) + energy quarters (4) + energy asymmetry (1)	**20**
Temporal burst and event patterns	timeout (1) + burst (7) + run-length (24) + first-order diffs (3)	**35**
Multi-scale variability across time	multi-scale chunk (6) + variance-ratio scale (3) + wavelet (14)	**23**
Periodicity and frequency structure	FFT (10) + AC at fixed lags (7) + AC decay (2)	**19**

Category	Sub-group	Count
Long-range dependence and persistence	Hurst (1) + trend slope (1)	**2**
Temporal complexity in low-intensity signals	approx / sample / permutation entropy (5)	**5**
Total		**104**

Plus 5 metadata columns (`label`, `run`, `window_start`, `path`, `baseline_rtt`) → **109 columns** in every `_features_fixed.csv`.